

Filosofía Ágil: Manifiesto ágil

Tiene origen el concepto de los procesos empíricos.

Proceso definido: PUD, tenemos una clara definición de las tareas a realizar. En el contexto del software tiene que ver con tener establecidas cuales son las actividades que se ejecutan y de qué manera. **Se sugiere que las salidas están controladas ante ciertas entradas** (Esto es válido si el entorno no es caótico y tiene ciertas condiciones)

Proceso empírico: Planteamos que la definición de . Nos permiten ajustarnos a un entorno caótico.

Es más una visión de hacia dónde vamos, pero no como llevarlo a cabo. Es una guía, o lineamientos. Entonces los frameworks de agile, esperemos que se alineen con estos valores.

Hay cuatro valores

- Individuo e interacciones sobre procesos (definidos) y herramientas (de los procesos).
 - Esto no quiere decir que tendremos procesos y herramientas, pero también se fomenta más la interacción
- Software funcionando por sobre documentación extensiva
 - No quiere decir que no documentamos.
- Colaborar con el cliente por sobre negociación contractual
 - Cuando las negociaciones son muy rígidas, puede que el contacto con el cliente se vea afectado.
- Responder al cambio sobre seguir un plan

Ciclo de proceso o ciclo de vida

Ciclo de proceso: Como y en qué orden se van a ejecutar las actividades del proceso

Doce principios del manifiesto ágil

Le dan un poco más de forma a los valores, sin llegar a prácticas concretas

Agile no es una metodología, ni un proceso, es una ideología, con principios definidos que guían el desarrollo del producto.

Que significa ágil?

Balance entre ningún proceso y demasiado proceso. La diferencia inmediata es la exigencia de una menor cantidad de documentación

Ser ágil no significa ser indisciplinado, o no seguir ningún plan, o no tener arquitectura

No es ir haciendo partecitas pequeñas sin ser funcional. Sino hacer software funcional y que nos sirva.

Requerimientos ágiles

Están guiados por los principios y valores ágiles. De ahí empezamos a descubrir los requerimientos.

Si hablamos de querer tener interacción cara a cara, que el software es la prioridad, los requerimientos en agile, estarán en busca de solucionar esos casos.

Se escriben de alguna forma en que se lean natural

Se determina lo que es solo “suficiente”, ya que muchas veces, hay requerimientos que no se usan, o se usan muy ocasionalmente, y nos implican pérdidas de esfuerzo.

Nosotros tenemos que responder de manera eficiente ante el cambio, tenemos que trabajar solamente en aquellos requerimientos que sean de alta prioridad.

En función de que le damos prioridad a los requerimientos?

- Por el valor que aportan ese feature cuando este implementado
- Los requerimientos se ordenan por prioridad del valor del negocio
- Trabajamos solamente con los requerimientos que están en la parte superior de la pila
 - Debemos conocerlos al detalle
 - Eso no implica que un requerimiento del fondo no pueda subir.

El mejor mecanismo de comunicación es cara a cara, permite que fluya información, y lo mejor es frente a un pizarrón.

Primer concepto de los requerimientos ágiles

Los requerimientos ágiles se focalizan en los Requerimientos de Negocio, y en los Requerimientos de Usuario

Segundo concepto de los requerimientos ágiles

Cuando trabajamos en la definición de un ciclo de vida iterativo-incremental, necesitamos definir en qué términos se va a ejecutar una determinada iteración.

En tradicional, cada iteración, abarca un alcance, y que requisitos, en el alcance dos, los otros requisitos, y en función del cronograma, se define los recursos y el tiempo. Cada iteración tiene tiempos diferentes.

En agile, el concepto es, como se dirige por valor, se define en cuánto tiempo y con qué recursos cuento, y a partir de eso, se deciden qué feature se incluyen en esa iteración. Ej: Cada cuatro semanas entrego sw funcional, y ahí se planifica que otros feature aplicare en las próximas 4 semanas.

User Stories

Partes

1. Tarjeta
 - a. Contiene una frase verbal, que define como algo concreto
 - b. Se agrega una prioridad y una estimacion
 - c. En el dorso de la tarjeta se pone la confirmacion.
2. Conversación: Parte mas importante.
3. Confirmación: Criterios de aceptacion, estan en el dorso de la tarjeta

Son multiproposito

- Identifica la necesidad del usuario.
- Una descripcion del producto
- En la pila de prioridad, las que estan arriba, deben estar escritos en mayor detalle.
Eso lo decide el Product Owner.

Cuando se implementa una user story, debe incluir todos los aspectos, base de datos, interfaz de usuario, backend. Esto no nos debe hacer perder de vista la arquitectura original, y es una parte del desafio.

Clase 24/04

Multiples niveles de planificacion

- Estrategia:
 - Que es lo que quiero cumplir
 - Es un objetivo organizacional
- Portfolio
 - Conjunto de productos o proyectos que se van a llevar a cabo
 - De todas las ideas que tengo par acumplir de una estrategia, cuales si, o cuales no
 - En que invertir? → Donde voy a gastar la plata
- Producto
 - Que voy a desarrollar, que voy a cumplir, cual es el alcance
- Release
 - Distintas etapas de un producto
 - A un MVP, se le empieza a agregar funcionalidades y corrigiendo errores o defectos que vamos a ir encontrando
 - Puede estar formado por varias iteraciones
 - Que y cuando entregar?
- Iteracion
 - Como se ira trabajando esos releases
 - Cada cierto tiempo se ira haciendo sprint en el cual se vera que desarrollar
 - Usualmente, conformada entre 15 a 30 dias

Cada nivel corresponde a una pregunta

Que y cuando estimamos?

Las estimaciones se vuelven mas precisas a medida que se baja de nivel

TIEMPO - ALCANCE - COSTO

SCRUM: El tiempo queda fijo, y lo que se modifica es el alcance

Si quiero agregar mas funcionalidad en ese tiempo, aumenta el costo

Planificacion de estrategia

- Es el nivel mas alto de planificacio
- Define hacia donde va la organizacion y que objetivos de negocio quiere lograr a largo plazo
- Incluye decisiones sobre mercados, clientes, valor
 - Si mi competencia saca algo mejor, yo debo superarlo

Planificación de portfolio

- En que productos o iniciativas invertir
- Buscar maximizar el valor del negocio
- Defino que incluyo y que no a través del dinero disponible

Planificación del producto

- Traduce la estrategia en una visión concreta del producto
- Es una solución concreta que genera valor al usuario
- Participan Product Owner, Stakeholders y Equipo SCRUM (Scrum master, desarrolladores)
- Se parte de una idea inicial y se definen recursos y restricciones
- Se genera vision, backlog priorizado y roadmap
- Permite organizar entregas futuras

Product backlog: Pila de users stories, se estiman solo las que entran al sprint.
A ese listado se dividen en releases.

Luego, a los releases se dividen en distintos sprints

Planificacion de Release

- Define que funcionalidades se entregaran en un periodo
- Inputs:
 - Backlog
 - Vision y velocidad del equipo
- Output:
 - Plan de release: Que funcionalidades y cuando

Diferencia entre verificacion de producto y verificacion de release

- Release
 - Refinamiento del product backlog, es decir, tomamos US que no sean spikes o epicas, y planificamos el release con esas US. Puede pasar que tambien

note que necesito otras US antes que no me habia dado cuenta cuando hice el product backlog. El refinamiento tambien incluye la division de las US en pequeñas tareas.

- Para hacer un release siguiente, se tiene en cuenta el anterior.
- Producto

Cadencia de los sprints: Cada cuanto se entrega valor

- En PUD, el sprints finaliza cuando se llega a una “x” cantidad de funcionalidades, sin tener en cuenta el tiempo
- Los releases plantean que puede ser:
 - Luego de multiples sprints
 - Luego de un sprints
 - Luego de cada feature → Luego de hacer una funcionalidad o US extra.

Planificacion de Iteracion: Sprint planning

- Primera actividad de cada sprint

Planificacion diaria: Daily

- Reunion diaria de 15 minutos
- Sirve para inspeccionar progreso y ajustar el plan
- Permite detectar bloqueos

Testing de software o prueba de software

Gestión Tradicional

Dentro de la gestión tradicional, en el contexto de asegurar la calidad, el testing es una de las actividades que está ligada a asegurar la calidad

El objetivo del testing no tiene que ver con asegurar la calidad del producto, en la gestión tradicional es un proceso destructivo, cuyo único objetivo es encontrar defectos (cuya existencia se asume). El testing exitoso es aquel que encuentra defectos.

La existencia de defectos es algo que se asume en el software, el SW en sí mismo contiene defectos.

En términos de costo del sw, un 30 o 50% se nos va en el esfuerzo del testing.

Busca asegurar la calidad, pero un buen testing no asegura la calidad de mi producto. La calidad del producto va acorde a como se construyó. El testing no nos puede asegurar calidad en el proceso de sw, ni sw de calidad, pero nos garantiza aumentar la confiabilidad.

Error vs Defecto

Error:

Defecto:

El testing busca defectos, porque se hace posterior a la etapa de desarrollo, salvo en el caso particular de testing unitarios, que ahí sí encuentra errores

Cuando hablamos de testing, hablamos de defectos, no de errores

No todos los defectos se tratan de la misma manera, y asumimos que en el SW esos defectos existen. No todos los defectos tienen la misma severidad

Severidad: Analiza el defecto de manera aislada

1. Bloqueante: A partir de encontrar ese defecto, no puedo continuar con la ejecución
2. Crítico: El resultado con ese defecto, es un resultado erróneo o no esperado, e implica un desvío importante
3. Mayor:
4. Menor: Un error con el que se asume que se puede seguir trabajando
5. Cosmético: Error muy leve

Prioridad: La prioridad tiene que ver con las necesidades del negocio, no con su severidad, nos determina cuál es el impacto de ese defecto en ese negocio.

1. Urgencia
2. Alta
3. Media
4. Baja

Niveles de prueba: Que es lo que quiero testear

- Pruebas unitarias
 - Testeo de los componentes individuales
 - Es la prueba que primero se automatiza
 - Se ejecuta por el mismo desarrollador
 - Es casi una actividad del desarrollo
 - Probamos de forma independiente, en el momento en que se encuentra (en este caso el error, ya que se esta construyendo), se resuelve en el momento
- Pruebas de integración
 - Hacer pruebas en donde los componentes probados de manera aislada, interactúan entre si en un ambiente controlado
 - Verificar que las partes que funcionan de manera independiente, funcionen bien en conjunto
 - Siempre debe ser incremental, los voy integrando de a poco y encontrar puntos mas visibles en los defectos
- Pruebas de sistema
 - Pruebas desde una perspectiva de usuario final, pero dentro de las areas tecnicas
 - Pruebo que el sistema funcione como un todo, se trabaja ya en un entorno donde se puede usar el sw, pero no es entorno de produccion, pero se busca el entorno mas parecido
 - Ya se tiene en cuenta requerimientos funcionales y no funcionales
- Pruebas de aceptacion
 - Pruebas que hace los usuarios
 - Aqui irian, en un contexto Agile, las pruebas en las users
 - Pruebas que realiza el usuario o los usuarios elegidos
 - Encontrar defectos no es el foco de estas pruebas, sino que la aplicacion haga lo esperado

Ambientes

- Desarrollo
 - Donde trabaja el equipo de desarrollo, se ejecutan las pruebas unitarias
- Prueba
 - Empiezan a trabajar los tester, con las pruebas d elos niveles de integracion
- Pre-produccion
 - Pruebas de sistema
- Produccion
 - Pruebas de aceptacion

Para poder ejecutar las pruebas, y entender como funciona el testing, necesitamos conocer el concepto de Caso de Prueba

Lo que buscamos, es escribir para el tester, cuales son las condiciones los pasos que tiene que seguir para ejecutar el sw, y cuales son los resultados que espero, producto de ejecutar el sw.

Es importante para guiar al tester, y si el tester identifica el efecto, debe identificar como reproducirlo, para poder subsanarlo. Nos permite reproducir el defecto, ya que tengo la secuencia de pasos y cuales son las variables contextuales con las que se ejecuto el programa para encontrar el defecto.

Definicion de casos de prueba

Probar exhaustivamente, es demasiado costo en recursos y tiempo, entonces se usan tecnicas, que intentan asegurar la mayor cobertura posible, con la menor cantidad de casos de prueba, abarco la mayor parte del sw.

Los bugs o defectos se encuentran en los valores extremos, o limites de los casos de prueba.

Derivación de casos de prueba

- Documentos del cliente
- Información de relevamiento
- Desde requerimientos: Si los requerimientos son ambiguos, los casos de prueba son más complicados. Si los requerimientos son claros, los casos de prueba son más sencillos
- Desde especificaciones de programación
- Desde el código

Los casos de prueba se pueden empezar a desarrollar desde que los requerimientos estan listos. No hace falta que este el codigo listo.

Condiciones de prueba

Ademas de los Casos de Prueba, las condiciones de prueba son los resultados esperados del caso de prueba frente a valores concretos que le asigno a ese caso de prueba.

Ej: Registrar cliente

CP:

El usuario con el perfil x, ingresa a la opcion registrar cliente

El usuario ingresa el cuit del cliente.

Condiciones de Prueba (tiene valores concretos)

Se prueba con el CUIT existente X

Si se busca que el cuit ya este repetido, en las Condiciones de Prueba, deberia especificarse con que cuit probar y eso (Reveer)

Partes del caso de prueba

1. Objetivo: Caracteristica del sist a comprobar
2. Datos de entrada y de ambiente: Datos a introducir al sistema que se encuentra en condiciones preestablecidas
3. Comportamiento esperado: La salida o la accion esperada en el sistema de acuerdo a los requerimientos del mismo

Estrategias

Buscan tener la mayor eficiencia al menor costo.

- Estrategia de caja negra
 - Ejecutar el software y definir frente a ciertas entradas, cuales son los resultados que se espera, y ver si coinciden. Pero no me importa que hace el software por dentro
 - Basado en especificaciones: Mas estructurado, donde defino cuales son los casos de prueba que me dan la mayor cobertura posible.
 - Particion de equivalencias
 - Analisis de valores limites
 - Basados en la experiencia: Se necesita un tester muy experimentado, que conoce el sw, y sabe por donde ir a buscar los defectos.
 - Como hago para la menor cantidad de entradas, tengo la mayor cobertura posible.
 - Adivinanza de defectos
 - Testing exploratorio
- Estrategia de caja blanca
 - Definir casos de prueba, pero a partir del codigo, para asegurarme que paso por todos los lugares del codigo, para encontrar la mayor cantidad de defectos posibles
 - Analiza la estructura del software, analizo el software por dentro
 - Se garantiza el testing coverage
 - No se analiza entradas ni salidas, se ve el codigo
 - Ej:
 - Cobertura de enunciados o caminos basicos
 - Cobertura de sentencias
 - Cobertura de decision
 - Si quisiera hacer cobertura de decision, busco los if, y a partir de esas decisiones, elaboro casos de prueba para que pase por todas esas decisiones.
 - Cobertura de condicion
 - Cobertura de decision/condicion
 - Cobertura multiple

Las dos tecnicas son complementarias, en caja negra me paro en lado del negocio. En caja blanca se piensa como ejecutarlo por el codigo, como hago para que las entradas pasen por todos los lugares

Ciclo de pruebas

Si tenemos definido todos los casos de prueba a ejecutar, cuando se ejecuta una vez, es un ciclo de test. Esos casos de prueba pueden arrojar defectos

El ciclo de prueba es la ejecución de la totalidad de los casos de prueba, aplicados en la misma version del sistema a probar.

Pero hay un tema. Se ejecuta un ciclo de prueba y corrijo los defectos. La correccion del defecto en si misma, puede generar nuevos defectos donde antes no los tenia.

Test de Regresión

Volver a ejecutar una prueba del sistema completo, y no solamente donde se corrigieron los defectos. La corrección del defecto, puede generarme un nuevo defecto.

Los primeros ciclos de prueba, se ejecutan con test de regresión

No todas las pruebas son automatizables.

Proceso de pruebas

Como aún estamos en una gestión tradicional, el proceso de testing tiene un conjunto de actividades definidas.

Las pruebas se planifican.

- Identificar todos los casos de pruebas: Se puede hacer cuando los requerimientos están identificados
- Ejecución de los casos de prueba:
- Analisis de fallas
- Identificar los roles
- Identificar el nivel de cobertura que vamos a tener. Su definición tiene impacto en el costo del software, cuanto mas cobertura tenga, mas caro sera. Lo que hay que acordar es un nivel que lo deje tranquilo, y que compense el esfuerzo vs beneficio.

El fin de las pruebas se da acuerdo a su criticidad y su prioridad a determinar con el cliente.

EN AGILE, EL TESTING NO ES ENCONTRAR DEFECTOS, SINO QUE EL SW SE CONSTRUYE ENTRE TESTER Y DESARROLLADORES PARA QUE NO HAYA DEFECTOS

Testing y ciclo de vida

Verificación y validación

Validación: Que se construya el sw correcto. Valido los test de aceptación con los requerimientos.

Verificación: Válido que se construya correctamente

Mitos

El testing no tiene que ver con la calidad del producto ni con la calidad del proceso, el testing no lo asegura. No es una etapa que comienza cuando se termina de programar, es un proceso que se empieza apenas están los requerimientos.

El tester es el enemigo del programador

Cuando el testing es suficiente?

- El testing exhaustivo es imposible
- Decidir cuanto testing es suficiente depende de la evaluación del nivel del riesgo y de los costos del proyecto
- Usamos los riesgos para determinar que testear primero

Cuanto testing es suficiente

El criterio de aceptación es lo que comunmente se usa para resolver el problema de determinar cuando una determinada fase de testing ha sido completada

Puede ser definido por:

- Costos, si consume determinada cantidad de horas, dejen de probar
- Porcentajes de test corridos sin fallas, de todo el ciclo de prueba, cuando no ha tenido defectos
- Fallas predichas aun permanecen en el software
- No ha defectos de una determinada severidad en el software

Clase 12/05: Practico

- Defecto != Error
- El testing agil, esta mas orientado a encontrar errores
- El testing tradicional, esta mas orientado a encontrar defectos
- Distintos niveles de testing
 - Unitarias
 - Integracion
 - De usuario
 - De aceptacion
- Cuatro ambientes
 - Desarrollo
 - PreProduccion
 - Produccion
- Estrategias:
 - Caja blanca
 - Caja negra
 - Planteamos ciertas entradas
 - Una secuencia de pasos
 - Obtenemos resultados
 - Comparamos entradas y resultados
 - Si son iguales, pasa la prueba
 - Sino, se debe volver a desarrollar para volver a probar
 - Dos partes
 - Condiciones externas:
 - Clases de equivalencias: Subconjunto de valores, que si hacemos una prueba, el resultado debe ser el mismo. Se identifican por los criterios de aceptación
 - Clases de equivalencia validas
 - Clases de equivalencia invalidas
 - Casos de prueba: Se hacen en base a las pruebas de usuario, va a haber un caso de prueba, por lo menos, por cada prueba de usuario

Practico 8

ENTRADA

Condiciones Externas	Clases Equivalencia Válida	Clases Equivalencia Invalida
Estados del Taxi Nombre, porque es lo que puede seleccionar	1. Libre 2. Solicitado 3. Fuera de servicio 4. Ocupado Son 4 clases de equivalencia distintas porque de acuerdo al estado, cambia el comportamiento. Son cuatro clases de equivalencia para una misma condición externa.	5. Cualquier otro estado No lo puede ingresar el usuario, así que puede tranquilamente no ir.
Sistema de ubicación	6. Que esté habilitado	7. Que esté inhabilitado
Posicionamiento del mouse	8. Posición del mouse	
Barrio	9. Barrio existente	10. Barrio inexistente
Chapa	11. Patente registrada con formato: XXX000 12. Patente registrada con formato: XX000XX	13. Patente con formato incorrecto 14. Patente inexistente
Usuario	15. Usuario con permisos de administrador	16. Usuario con cualquier otro permiso

SALIDA

Condiciones Externas	Clases Equivalencia Válida	Clases Equivalencia Invalida:
		En las salidas, lo invalido serian los mensajes de error, o errores
Taxi ocupado	17. Muestra - la información del pasajero - muestra el estado del taxi - muestra la fecha y hora de inicio, costo del viaje Es una sola, porque se muestra todo junto cuando se pasa el cursor sobre un taxi ocupado	
Taxi solicitado	18. Muestra los datos del pasajero	

Colores estado	19. Verde, Amarillo, Rojo, Negro	
Errores		20. Sistema de localización desactivado 21. Taxi inexistente 22. Barrio Inexistente 23. Chapa con formato incorrecto No significan fallos de ejecucion, significa que no se logro el resultado esperado de la user

Posibles salidas: Mail con su formato, alertas, reportes, estadísticas

Hay un mensaje de error como minimo, por cada clase de equivalencia invalida delas entradas

Casos de prueba

Visualizar taxis libres para un barrio existente

Un caso de prueba esta formado de la siguiente manera:

- Numero para identificarlo
- Prioridad
- Nombre descriptivo
- Clases de equivalencia
- Precondiciones
- Pasos
- Resultado esperado (PASA o FALLA)

Num id	Prioridad	Nombre descriptivo	Clases eq	Precond.	Pasos	Res
1	Alta	Ver taxis libres para un barrio existente	1, 6, 8, 9, 12, 15, 19, 25	-El usuario "Joaquin" debe tener permiso de administrador -Sistema de localización activado y configurado en la ciudad de Córdoba -El barrio "Alberdi" debe estar cargado en el sistema -Taxi libre cargado en el sistema con patente AAA-000 conectado en el sistema de Localización en "Alberdi" -Debe existir el estado libre	1.Joaquin Selecciona ver mapa filtrado 2.Joaquin selecciona el barrio alberdi 3.Joaquin selecciona el estado libre 4.Joaquin confirma la búsqueda	5.Se muestra el mapa con un icono verde, que contenga la informacion del taxi patente: AAA000, color: gris, model: fluence, marca: Renault

2	Baja	Visualizar taxi solicitado en barrio inexistente	6, 2, 10, 12, 15, 25, 19,22	-Joaquin debe tener usuario de administrador -Geolocalizador activo y configurado -Barrio “Alberdi” cargado en base de datos -Taxi cargado con chapa AA000AA en estado solicitado cargado en el sistema -Pasajero “Francisco” solicito el taxi AA000AA -Debe existir el estado solicitado	1.El usuario Joaquin selecciona ver mapa de taxis 2.Joaquin selecciona “Barrio Jardin” 3. Joaquin filtra por estado solicitado 4.Joaquin confirma la busqueda	1.El sistema muestra el formulario de busqueda 4.El sistema muestra el mensaje de error: “Barrio ingresado inexistente”

Prioridad alta: Generalmente los que van asociados al PASA, casos de exito

En un caso de prueba de exito, van todas las VALIDAS

Si el ultimo paso termina en 4, el resultado debe ser 4 o 5, pero no puede ser 1, para seguir un hilo.

No vimos valores limites, si hay valores limites, en las precondiciones, pasos, debe probarse con los valores del limite. Siempre que se especifiquen rangos, fechas limites, algun minimo, o algun maximo, se debe probar con esos datos.

Clase 15/05: Teorico

Lean es una filosofía que piensa en la mejora de un proceso continuo. No esta pensando en algo que empieza y termina

KANVAN plantea que hay que sostener lo que tenemos, e irlos mejorando poco a poco

Tipos de desperdicios

- Tipo 1: Desperdicios necesarios
- Tipo 2: Puro desperdicio

7 + 1 Desperdicios

- Talento no usado: Si tenemos personal capacitado y no lo usamos, es un desperdicio
- Movimiento
- Espera
- Sobreprocesamiento

Los 7 desperdicios LEAN (en servicios basados en conocimientos)

Donde la creacion de SW es un servicio basado en conocimiento

Desperdicios en manufactura	Desperdicios en servicios basados en conocimiento
Inventario	Trabajo parcialmente terminado: No cuenta, pero se gasto recursos en intentarlo
Sobreproduccion	Caracteristicas extra, hacer algo por las dudas, ya que estamos
Procesos extra	Procesos extra, no se agreguen tareas que no sumen valor
Transporte	Cambio de tareas, transferencia de conocimiento. No hay que distraerse, comenzar tareas y finalizarlos
Espera	Esperas / Demoras. Hacer una cosa a la vez, si uno hace mas cosas en simultaneo, esta demostrado que se demora mas. Hay que focalizarse en terminar y no empezar mas cosas sin terminar
Movimiento	Cambio de contextos
Defectos	Defectos
Talento no utilizado	Talento no utilizado

KANBAN

Es un método para definir y gestionar y mejorar servicios que entregan trabajo del conocimiento tales como servicios profesionales, trabajos o actividades en las que interviene la creatividad y el diseño de productos de software o físicos

Es un método para la mejora de procesos

Para gestionar servicios (NO PROCESOS, COMO EN SCRUM, es menos prescriptivo que SCRUM incluso). Kanban se usa en el contexto de un proceso que tiene que continuar.

Su origen se dio a fines de 1940, donde Toyota comenzó a estudiar técnicas de almacenamiento y tiempo de stock en los supermercados.

La primer premisa es que si la autopista está al 100% de su capacidad, el flujo se frena. No se puede seguir un proceso si esta al 100% de la capacidad.

Valores de kanban

- Transparencia
- Equilibrio
- Colaboración
- Foco en el cliente
- Flujo
- Liderazgo
- Entendimiento
- Acuerdo
- Respeto

SER ÁGIL: Seguir los valores y principios de la filosofía ágil

HACER AGIL: Tiene que ver con las prácticas que se hacen en las filosofías ágiles

Principios de Kanban

- Gestión de cambios
 - **Comenzar con lo que haces ahora:** Primero se identifican las actividades de ese proceso, y trabajar con esas actividades, no se cambian. Se ve como mejorar lo que se hace
 - **Acordar la búsqueda de la mejora a través del cambio evolutivo.** Mejoramos lo que se hace, pero a través del aprendizaje. Yo veo que tenemos hoy
 - Fomentar actos de liderazgo a todos los niveles: Relacionado con la autonomía de un equipo. En todos los equipos se fomenta el liderazgo
- Entrega de servicios
 - Comprender y enfocarse en cumplir las necesidades y expectativas del cliente
 - Gestionar el trabajo, dejar que los trabajadores se autoorganicen en torno a el
 - Revisar periódicamente la red de servicios y sus políticas para mejorar los resultados entregados

Practicas generales de KANBAN

Las practicas tienen que ver con cuestiones concretas del trabajo a hacer

- **Visualizar:** Todos los integrantes deben ver lo que se hace y como se hace. Un modelo visual del proceso y de lo que se esta haciendo. Ver que hace cada miembro del equipo
- **Limitar el trabajo en curso:** Dejar de empezar, y empezar a terminar. Nadie puede hacer tantas cosas al mismo tiempo.
- **Gestionar el flujo:** Se necesita que las piezas de trabajo fluyan, no pueden estar tan sobrecargados. Por eso tambien es importante limitar el trabajo en curso
- **Hacer las políticas explícitas:** La gente se debe autoasignar el trabajo, si no tengo nada para hacer, busco algo. Pero deben estar explícitas como hacer esa asignación, cuantas piezas pueden estar en curso, etc. Deben ser pocas las políticas, pero deben determinar las pautas de trabajo.
- **Establecer ciclos de retroalimentación:**
- **Mejorar colaborativamente, evolucionar experimentalmente**

Visualizar

- Permite absorber y procesar una gran cantidad de informacion en un corto intervalo de tiempo
- Habilita la cooperacion, ya que todo el mundo tiene la misma imagen
- Permite tomar decisiones de una manera mas rapida y colaborativa
- Hay columnas de espera, para limitar el trabajo en curso. Ej: Si hay una tarea terminada y debe pasar a testing, y el tester tiene una tarea asignada, debe esperar.

Limitar el WIP provoca conversacion y mejora

El WIP es cuantas piezas vamos a tener ejecutando una tarea en simultaneo. Se hace generalmente para comenzar:

Cantidad de recursos - 1

Ya que no tienen que estar los recursos a 100%

Para cada una de las actividades que tenemos definidos en el proceso, tenemos un limite del trabajo en progreso.

Ayudar a que el trabajo fluya

Al 100% de capacidad, el trabajo realizado es minimo. Esto pasa cuando el WIP en cada area coincide con la cantidad de recursos

Limitar el trabajo en progreso

- Los sistemas eficaces son los que se centran en el flujo de trabajo y menos en tener ocupados a los trabajadores
- Cuando los recursos estan ocupados completamente, no hay holgura, si algo se estanca, queda bloqueado el flujo.

Limite WIP (Work in progress)

Asignar limites explicitos de cuantos items puede haber en progreso en cada estado del flujo de trabajo

Nunca se deja tareas a la mitad, si necesitamos que un recurso se mueva a otra actividad, primero debe terminar su trabajo.

Administracion de colas

Como aplicar Kanban?

Dividir el trabajo en piezas. En un tablero siempre son piezas de trabajo, pueden ser requerimientos, bugs.

Tambien se pueden asignar porcentajes de cuantas piezas de que tipo van al tablero (Kanban no dice cuanto de cada uno, lo asigna el mismo proceso o el equipo)

Hacer explicitas las politicas

Todas las politicas deben ser acordadas entre todas las partes involucradas, con los clientes, interesados y trabajadores

Las politicas deben estar expuestas en un lugar destacados

Deben ser:

- Pocas
- Sencillas
- Bien definidas
- Visibles
- Aplicables en todo momento
- Modificables por los que prestan el servicio

Mejorar colaborativamente

Como equipo, y viendo como fluye el trabajo, y al ser equipo autogestionado, y vamos viendo como podemos mejorar. El ir viendo como mejorar, esta relacionado con el metodo cientifico, planteamos una hipotesis, y verificamos. Por ej: Aumentar el WIP en un area, puede mejorar el flujo. Y se verifica

Es decir, podemos ir planteando hipotesis y verificando en pos de mejorar, si no sirve, se vuelve hacia atras.

La retroalimentacion en KANBAN, las reuniones o los momentos de feedback estan en las Cadencias. Nosotros como equipo definimos cada cuanto haremos releases, planificaciones y revisiones, y ese tiempo debe ser fijo.

Recordar que KANBAN no es de un proyecto, por lo que no es uniforme, no es de un unico producto. Puede haber de todo.

Métricas

Son la base para definir los SLA. Recordar que el concepto del agilismo con las métricas es una filosofía minimalista. Son pocas métricas, mido solamente lo que me interesa

- Lead time: Vista del cliente, es el tiempo que transcurre desde que el cliente me pide algo hasta que se lo entrego
- Cycle time: Vista interna, es el tiempo que transcurre desde que empieza la actividad hasta que se lo entrego al cliente.
- Touch time: Tiempo de tocado, es el tiempo que efectivamente estuvimos trabajando en el requerimiento. Se intenta que el Touch time y el Cycle time sean lo mas parecido posible, porque eso indica que no hay cuellos de botellas. Si hay mucho tiempo de diferencia, puede significar desperdicios, que el flujo se atasca.

Las métricas están orientadas a servicios

- Me permite saber la capacidad de nivel de servicio al que el sistema puede entregar

NO ENTRA PRACTICAS CONTINUAS AL PARCIAL.

Clase 19/05: Practico

Comprar entrada

ENTRADA

Condicion Externa	Clase Equivalencia Validas	Clase Equivalencia Invalidas
Fecha	1. Fecha actual o futura menor a 10 días en rango de parque abierto	2. Fecha pasada 3. Fecha fuera de rango 4. Fecha vacía
Cantidad de entradas	5. Cantidad de entradas menor o igual a 10 y mayor a 0 Es importante ver que cuando hay rango va en una sola clase de equivalencia.	6. Cantidad de entradas mayor a 10 o menor o igual a 0 7. Cantidad nula
Tipo de pase	8. VIP 9. Regular	10. Tipo de pase nulo
Usuario registrado	11. Esta registrado con perfil	12. No esta registrado 13. No tiene perfil
Forma de pago	14. Efectivo 15. Tarjeta	16. Forma de pago nulo

SALIDA

Condicion Externa	Clase Equivalencia Validas	Clase Equivalencia Invalidas
Mail confirmacion	17. Mail confirmacion	
Redireccionamiento a Mercado Pago	18. Se redirecciona a mercado pago para solicitar los datos de la tarjeta	
Cantidad de entradas compradas y fecha de visita	19. Se muestra la cantidad de entradas compradas y la fecha de visita	
Errores		

Num id	Prioridad	Nombre descriptivo	Clases eq	Precond.	Pasos	Res
1	Alta	Comprar entrada regular con mercado pago con fecha actual dentro del rango	1, 5, 9, 11, 15, 17, 18, 19	-El usuario "Joaquin" esta registrado y logueado en el sistema con su mail asociado joaquin@gmail.com -Tipo de pase "Regular" cargado en la base de datos -Forma de pago "Tarjeta" cargado en la base de datos (Se escribe asi o se escribe: Debe existir la forma de pago "Tarjeta" en el sistema) -Fecha actual 19/05/2026	1. Joaquin selecciona Comprar Entradas 2. Joaquin ingresa la fecha 19/05/2026 3. Joaquin ingresa cantidad de entradas = 1. 4. Joaquin ingresa la edad de 25 años 5. Joaquin selecciona el tipo de pase "Regular" 6. Joaquin selecciona la forma de pago "Tarjeta" 10. Joaquin confirma la compra	5. se muestra el monto total. 6. Se redirecciona a Mercado pago y se logra exitosamente 7. Se manda confirmacion de mail a joaquin@gmail.com 8. Se muestra cantidad de entradas "1" y fecha 19/05/2026

Inscribir a actividad
ENTRADA

Condicion Externa	Clase Equivalencia Validas	Clase Equivalencia Invalidas
Actividad a realizar	1. Actividad registrada con cupo	3. Actividad no registrada

	disponible que requiere vestimenta 2. Actividad registrada con cupo que no requiere vestimenta.	4. Actividad registrada sin cupo
Horario de actividad	5. Horario habilitado	6. Horario no habilitado
Cantidad visitantes	7. Cantidad positiva menor al cupo disponible	8. Cantidad menor o igual a 0 9. Cantidad mayor al cupo disponible
Aceptacion terminos y condiciones	10. Acepta T y C	11. No acepta T y C
Nombre	12. Nombre válido	13. Nombre vacío 14. Nombre con numeros
DNI	15. DNI numero con 8 digitos	16. DNI invalido 17. DNI vacio
Edad	18. Edad mayor o igual a 1 19. Edad menor o igual a 99	20. Edad menor a 1 21. Edad mayor a 99
Talle	22. Si 23. No	

SALIDA

Condicion Externa	Clase Equivalencia Validas	Clase Equivalencia Invalidas
Mail	Mail enviado al usuario con los datos de la actividad	
Errores		Campos vacíos Formato dni invalido Formato nombre invalido No acepto términos y condiciones

Clase 22/05: TDD - Test Driven Development

Primero se hacen las pruebas, luego la programación

“El acto de diseñar tests es uno de los mecanismos conocidos más efectivos para prevenir errores...El proceso mental que debe desarrollarse para crear tests útiles puede descubrir y eliminar problemas en todas las etapas del desarrollo”

B. Beizer

“Test-Driven Development”: Kent Beck. XP

¿Por qué se busca hacer las pruebas primero, previo a empezar a hacer el código?

- Para pensar que el código no funciona de manera aislada, no son partes aisladas entre sí. El código tiene que levantar un servidor, tiene que conectarse a base de datos. Si nosotros no hacemos un código que fue testeado, es difícil encontrar los defectos



El tdd es proactivo, busco desarrollar las pruebas, y luego hacer el código en base a las pruebas hechas.

Las tres leyes de TDD



No escribir una línea de código hasta que no hayas escrito antes un test case que falla

Si no hay un test que justifique una función, esa función no debería existir



No es necesario escribir más test de los necesarios para que falle. Normalmente con un test que falle es suficiente.

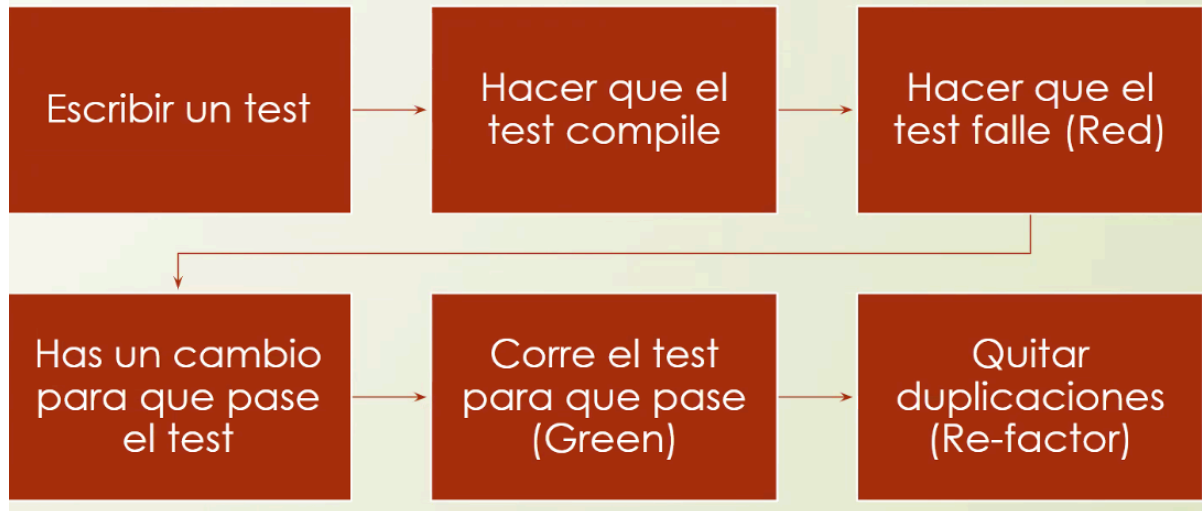
No hace falta hacer muchas pruebas de golpe, si con 1 o 2 ya podemos justificar la función que queremos desarrollar



No vas a escribir más código en Producción que el necesario para que el test pase.

Si tenemos un método que debe devolver VERDADERO por ej, el código de la función, debe ser el mínimo necesario para que se devuelva VERDADERO.

Pasos del Desarrollo conducido por Testing (TDD)



Visualizar kits de productos

Entradas

Condiciones Externas	CE Validas	CE Invalidas
Nombre de kit	1. Nombre de kit conformado por letras y números con resultado 2. Nombre de kit conformado por letras y numeros sin coincidencias 3. No ingresa nombre de kit	4. Nombre de kit con caracteres especiales (@ o algo asi)
Codigo de kit	5. Codigo de kit numerico entero mayor a 0 6. No ingresa codigo kit	7. Codigo de kit con formato decimal 8. Codigo de kit negativo
Nombre producto	9. Nombre de producto conformado por letras y numeros con resultado 10. Nombre de producto conformado por letras y numeros sin resultado 11. No ingresa nombre producto	12. Nombre de producto con caracteres especiales
Codigo de producto	13. Codigo de producto numerico entero mayor a 0 14. No ingresa codigo de producto	15. Codigo de producto con formato decimal 16. Codigo de producto negativo
Usuario	17. Usuario logueado con perfil de administrado	18. No tiene usuario logueado 19. Usuario logueado sin perfil de administrador
Formato de visualizacion	20. pdf 21. pantalla 22. excel	23. No selecciona formato de visualizacion

Salidas

Condiciones Externas	CE Validas	CE Invalidas
Visualizacion de kit de productos	24. Se mostrara nombre de kit y codigo de kit en formato pdf 25. Se mostrara nombre de kit y codigo de kit en pantalla 26. Se mostrara nombre de kit y codigo de kit en excel 27. Se mostrara un msj informando que no hay kits que coincidan con el filtro de busqueda	
Errores		

ID	Prioridad	Nombre Descriptivo	Clases de Equivalencias	Precondiciones	Pasos	Resultados
1	Alta	Visualizar kit de productos aplicando filtro de nombre de kit y visualizarlo con formato pdf	1, 6, 11, 14, 17, 20, 24	-El usuario "Joaquin Acosta" logueado y con permisos de "Administrador" -Kit "brilli brilli", con codigo "123ab" cargado en base de datos con los siguientes productos: "Nombre: Pintura azul, codigo: 33", "Nombre:Brillantina, codigo: 22" -Formato de visualizacion "PDF" cargado en la base de datos	1. El usuario Joaquin Acosta ingresa a la opcion "Visualizar kits" 2. El usuario Joaquin Acosta ingresa el nombre de kit "brilli brilli" 3. El usuario Joaquin Acosta no ingresa codigo de kit 4. El usuario Joaquin Acosta no ingresa nombre de producto 5. El usuario Joaquin Acosta no ingresa codigo de producto 6. El usuario Joaquin Acosta selecciona la opcion "PDF" para visualizar los kits 7. El usuario Joaquin Acosta confirma los filtros de busqueda	8. Se muestra el kit "brilli brilli" y su codigo 123ab en formato pdf

US: Generar cupon de descuento

Entrada

Condiciones Externas	CE Validas	CE Invalidas
Usuario	1. Usuario logueado con rol de Vendedor	2. Usuario logueado con rol distinto a Vendedor 3. No hay usuario logueado
Cliente	4. Cliente seleccionado	5. No se seleccionan clientes
Tipo de descuento	6. Descuento por porcentaje 7. Descuento por monto fijo	8. No se selecciona tipo de descuento
Codigo de cupon NO VA	9. Codigo numerico unico	10. Codigo con formato distinto 11. Codigo repetido
Validez de la oferta	12. Fecha desde y fecha hasta con formato dd/mm/aaaa	13. Fechas con formato invalido 14. Fecha desde mayor a fecha hasta 15. No se carga fecha desde 16. No se carga fecha hasta 17. No se cargan fechas de validez de oferta
Descuento	18. Numero entre 0 a 100 para indicar Porcentaje de descuento si se seleccion tipo de descuento por porcentaje 19. Numero mayor a 0 para indicarMonto de descuento si se selecciono tipo de descuento por monto fijo	20. No se ingresa ningun porcentaje de descuento 21. Se ingresa un porcentaje mayor a 100 22. Se ingresa un porcentaje menor a 0 23. No se ingresa ningun monto de descuento 24. Se ingresa un monto negativo

Salida

Condiciones Externas	CE Validas	CE Invalidas